

S R P

Single Responsibility Principle

One class, one reason to change

DEFINITION

A module should have one reason to change — it answers to a single actor or stakeholder.

A "reason to change" is an actor — a person or role who requests it: the CFO's accounting team, the COO's operations team, the DBA. SRP says gather what one actor cares about into one place, and split what different actors care about. The seam you cut follows the org chart, not the technology.

WHY IT MATTERS

When two actors share a class, a change one of them asks for can silently break the other's behavior — and two teams end up editing the same file, colliding in reviews and merges. SRP shrinks the blast radius: each actor's logic lives in one place, so an edit stays contained and you can tell who is affected just by reading the class boundaries.

— How it works

Actor

A source of change — a person or role who requests it (Accounting / CFO, Operations / COO, the DBA). Not a user of the system, but whoever asks for it to change.

Responsibility

A family of functions serving one actor. "One responsibility" means one reason to change — not one method. A class may have many methods if they all answer to the same actor.

Boundary

The seam between actors. When two actors would share a class, split along that line; if callers still want one door, a thin Facade can recombine the parts.

HOW TO APPLY

1. For each method, ask: which actor requests changes to it?
2. Group the methods by actor. If a class serves two actors, that is two responsibilities.
3. Extract each actor's methods into its own class (PayCalculator, HourReporter, EmployeeRepository).
4. If callers need one entry point, add a thin Facade that delegates — it recombines behavior without re-coupling the actors.

LITMUS TEST

For each method ask: which actor requests changes to this? If two different roles do, the class has two responsibilities — separate them.

— Smell → fix

One Employee class answers to three different actors at once:

```
class Employee {  
  calculatePay() { /* tax rules */ } // Accounting / CFO  
  reportHours() { /* timesheets */ } // Operations / COO  
  save() { /* SQL */ } // DBA  
}  
// one class, three reasons to change
```

↓ REFACTOR BY ACTOR

```
class Employee {} // shared data only  
class PayCalculator {} // Accounting owns this  
class HourReporter {} // Operations owns this  
class EmployeeRepository {} // DBA owns this
```

Each class now changes for one reason only. Need one entry point for callers? Add a thin EmployeeFacade that delegates to the three — it recombines behavior without re-coupling the actors.

USE WHEN

- ✓ Policy, persistence and formatting are tangled in one class
- ✓ Two teams edit the same file for different reasons

AVOID WHEN

- ▲ Splitting into anemic one-method classes with no cohesion

— Trade-offs & pitfalls

PROS

- + Localized, low-risk change
- + Smaller, focused tests

CONS

- Over-splitting scatters related logic
- More files to navigate

COMMON PITFALLS

- ▲ Splitting by method count instead of by actor — you get anemic one-method classes and lose cohesion.
- ▲ Reading "responsibility" as "does one thing" — a class can have many methods if they all serve one actor.
- ▲ Over-applying to a small script: three files for two trivial, never-changing duties is worse than one clear class.

WHERE YOU SEE IT

- ▶ A thin Controller that delegates to fat Services — UI concerns and business rules change for different reasons.
- ▶ A Repository that keeps persistence out of the domain entity — the DBA's schema and the domain's rules evolve apart.
- ▶ A React component split into presentation plus a custom hook — visual changes vs data-logic changes.

SEE ALSO

High Cohesion	the same idea in GRASP terms — keep related work together
SoC	SRP at class scale; Separation of Concerns is its architectural scale
ISP	apply the one-actor lens to interfaces, per client
OCP	small, single-purpose units are what you extend without editing

— Interview & sources

Does SRP mean one method per class?

No — one reason to change. A class can have many methods if they all serve the same actor.

What exactly is a "reason to change"?

An actor: the person or role that requests it (CFO, COO, DBA). Different actors = different responsibilities.

Isn't splitting everything just more files?

Over-splitting into anemic one-method classes hurts cohesion. Split by actor, not by line count; recombine behind a Facade if callers need one door.

How does SRP relate to the rest of SOLID?

It is the foundation: small, single-actor classes are what make OCP, LSP, ISP and DIP practical. Get the boundaries wrong and every other principle gets harder.

REMEMBER

Gather the things that change for the same reason; separate the things that change for different reasons.

SOURCES

Robert C. Martin — "Agile Software Development, Principles, Patterns, and Practices" (2003)

Robert C. Martin — "Clean Architecture" (2017): a module should be responsible to one, and only one, actor